

DBMS – ORDER AND ORDER ITEMS MANAGEMENT

Project Title: Etsy Handmade Marketplace Database Management System

1. Database Design

1.1 Orders Table

Attribute	Key Type
Order_ID	Primary Key
Customer_ID	Foreign Key
Order_Date	-
Order_Status	Check
Total_Amount	-

1.2 Order_Items Table

Attribute	Key Type
Order_Item_ID	Primary Key
Order_ID	Foreign Key
Product_ID	Foreign Key
Quantity	-
Price	-

2. Primary and Foreign Key Relationship

Customer (1) —places→ (M) Orders (1) —has→ (M) Order_Items (M) —refers to→ (1) Product, where Product (M) —listed by→ (1) Shop and Shop (M) —owned by→ (1) Seller.

PK = Primary Key FK = Foreign Key 1 = One M = Many

3. Table Creation

3.1 Create Orders Table

```
CREATE TABLE Orders (  
  Order_ID NUMBER PRIMARY KEY,  
  Customer_ID NUMBER NOT NULL,  
  Order_Date DATE NOT NULL,  
  Order_Status VARCHAR2(20),  
  Total_Amount NUMBER(10,2),  
  CONSTRAINT fk_order_customer  
  FOREIGN KEY (Customer_ID)  
  REFERENCES Customer(Customer_ID),  
  CONSTRAINT chk_order_status  
  CHECK (Order_Status IN  
  ('Pending', 'Confirmed', 'Shipped', 'Delivered', 'Cancelled'))  
);
```

3.2 Create Order_Items Table

```
CREATE TABLE Order_Items (  
  Order_Item_ID NUMBER PRIMARY KEY,  
  Order_ID NUMBER NOT NULL,  
  Product_ID NUMBER NOT NULL,  
  Quantity NUMBER NOT NULL,  
  Price NUMBER(10,2) NOT NULL,  
  Sub_total NUMBER(10,2),  
  FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID),  
  FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)  
);
```

4. Data Insertion

4.1 Orders Records

```
INSERT INTO Orders  
(Order_ID, Customer_ID, Order_Date, Order_Status, Total_Amount)  
VALUES  
(2001, 201, TO_DATE('22-09-2026', 'DD-MM-YYYY'), 'Pending', 1450);  
1 row created.
```

```
INSERT INTO Orders  
(Order_ID, Customer_ID, Order_Date, Order_Status, Total_Amount)  
VALUES  
(2002, 202, TO_DATE('21-09-2026', 'DD-MM-YYYY'), 'Confirmed', 890);  
1 row created.
```

```
INSERT INTO Orders  
(Order_ID, Customer_ID, Order_Date, Order_Status, Total_Amount)  
VALUES  
(2003, 203, TO_DATE('20-09-2026', 'DD-MM-YYYY'), 'Shipped', 1800);  
1 row created.
```

```
INSERT INTO Orders  
(Order_ID, Customer_ID, Order_Date, Order_Status, Total_Amount)  
VALUES  
(2004, 201, TO_DATE('19-09-2026', 'DD-MM-YYYY'), 'Delivered', 650);  
1 row created.
```

```
INSERT INTO Orders  
(Order_ID, Customer_ID, Order_Date, Order_Status, Total_Amount)  
VALUES  
(2005, 204, TO_DATE('18-09-2026', 'DD-MM-YYYY'), 'Cancelled', 420);  
1 row created.
```

```
INSERT INTO Orders  
(Order_ID, Customer_ID, Order_Date, Order_Status, Total_Amount)  
VALUES  
(2006, 205, TO_DATE('17-09-2026', 'DD-MM-YYYY'), 'Pending', 980);  
1 row created.
```

```
INSERT INTO Orders
(Order_ID, Customer_ID, Order_Date, Order_Status, Total_Amount)
VALUES
(2007, 206, TO_DATE('16-09-2026', 'DD-MM-YYYY'), 'Cancelled', 1350);
1 row created.
```

```
INSERT INTO Orders
(Order_ID, Customer_ID, Order_Date, Order_Status, Total_Amount)
VALUES
(2008, 201, TO_DATE('10-09-2026', 'DD-MM-YYYY'), 'Delivered', 1750);
1 row created.
```

```
INSERT INTO Orders
(Order_ID, Customer_ID, Order_Date, Order_Status, Total_Amount)
VALUES
(2009, 207, TO_DATE('11-09-2026', 'DD-MM-YYYY'), 'Shipped', 2600);
1 row created.
```

```
INSERT INTO Orders
(Order_ID, Customer_ID, Order_Date, Order_Status, Total_Amount)
VALUES
(2010, 209, TO_DATE('21-09-2026', 'DD-MM-YYYY'), 'Confirmed', 3200);
1 row created.
```

4.2 Order_Items Records

```
INSERT INTO Order_Items VALUES (1, 2001, 301, 1, 850, 850);
1 row created.
```

```
INSERT INTO Order_Items VALUES (2, 2002, 304, 1, 890, 890);
1 row created.
```

```
INSERT INTO Order_Items VALUES (3, 2003, 307, 2, 900, 1800);
1 row created.
```

```
INSERT INTO Order_Items VALUES (4, 2004, 303, 1, 650, 650);
1 row created.
```

```
INSERT INTO Order_Items VALUES (5, 2005, 309, 1, 420, 420);
1 row created.
```

```
INSERT INTO Order_Items VALUES (6, 2006, 302, 2, 490, 980);
1 row created.
```

```
INSERT INTO Order_Items VALUES (7, 2007, 308, 1, 1350, 1350);
1 row created.
```

```
INSERT INTO Order_Items VALUES (8, 2008, 305, 2, 875, 1750);
1 row created.
```

```
INSERT INTO Order_Items VALUES (9, 2009, 310, 1, 2600, 2600);
1 row created.
```

```
INSERT INTO Order_Items VALUES (10, 2010, 306, 2, 1600, 3200);
1 row created.
```

```
INSERT INTO Order_Items VALUES (11, 2001, 306, 1, 600, 600);
1 row created.
```

5. Data Retrieval

5.1 Display Orders

```
SELECT * FROM Orders;
```

ORDER_ID	CUSTOMER_ID	ORDER_DAT	ORDER_STATUS	TOTAL_AMOUNT
2001	201	22-SEP-26	Pending	1450
2002	202	21-SEP-26	Confirmed	890
2003	203	20-SEP-26	Shipped	1800
2004	201	19-SEP-26	Delivered	650
2005	204	18-SEP-26	Cancelled	420
2006	205	17-SEP-26	Pending	980
2007	206	16-SEP-26	Cancelled	1350
2008	201	10-SEP-26	Delivered	1750
2009	207	11-SEP-26	Shipped	2600
2010	209	21-SEP-26	Confirmed	3200

10 rows selected.

5.2 Display Order_Items

```
SELECT * FROM Order_Items;
```

ID	ORDER_ID	PRODUCT_ID	QTY	PRICE	SUB_TOTAL
1	2001	301	1	850	850
2	2002	304	1	890	890
3	2003	307	2	900	1800
4	2004	303	1	650	650
5	2005	309	1	420	420
6	2006	302	2	490	980
7	2007	308	1	1350	1350
8	2008	305	2	875	1750
9	2009	310	1	2600	2600
10	2010	306	2	1600	3200
11	2001	306	1	600	600

11 rows selected.

6. Order and Order_Items Modification

6.1 Update Order Status

```

UPDATE Orders
SET Order_Status = 'Confirmed'
WHERE Order_ID = 2001;
1 row updated.

UPDATE Orders
SET Order_Status = 'Shipped'
WHERE Order_ID = 2002;
1 row updated.

SELECT * FROM Orders WHERE Order_ID IN (2001, 2002);

```

ORDER_ID	CUSTOMER_ID	ORDER_DAT	ORDER_STATUS	TOTAL_AMOUNT
2001	201	22-SEP-26	Confirmed	1450
2002	202	21-SEP-26	Shipped	890

6.2 Update Order Quantity / Price

```

UPDATE Order_Items
SET Quantity = 2
WHERE Order_Item_ID = 1;
1 row updated.

UPDATE Order_Items
SET Price = Price + 50
WHERE Order_Item_ID = 3;
1 row updated.

UPDATE Order_Items
SET Quantity = 3
WHERE Order_Item_ID = 10;
1 row updated.

SELECT * FROM Order_Items WHERE Order_Item_ID IN (1, 3, 10);

```

ID	ORDER_ID	PRODUCT_ID	QTY	PRICE	SUB_TOTAL
1	2001	301	2	850	850
3	2003	307	2	950	1800
10	2010	306	3	1600	3200

commit;

7. Customer-Order History Report

The output displays each customer's order history, including order ID, order date, product ID, quantity, price, and subtotal. It helps track the items purchased by each customer and the details of their Etsy orders.

```

SELECT
c.Customer_ID,
c.Customer_Name,
o.Order_ID,

```

```

o.Order_Date,
oi.Product_ID,
oi.Quantity,
oi.Price,
oi.Sub_total
FROM Customer c
JOIN Orders o
ON c.Customer_ID = o.Customer_ID
JOIN Order_Items oi
ON o.Order_ID = oi.Order_ID
ORDER BY c.Customer_ID, o.Order_Date;

```

CUST_ID	CUSTOMER_NAME	ORDER_ID	ORDER_DAT	PRODUCT_ID	QTY	PRICE	SUB_TOTAL
201	Emma Wilson	2008	10-SEP-26	305	2	875	1750
201	Emma Wilson	2004	19-SEP-26	303	1	650	650
201	Emma Wilson	2001	22-SEP-26	301	2	850	850
201	Emma Wilson	2001	22-SEP-26	306	1	600	600
202	Sophia Clark	2002	21-SEP-26	304	1	890	890
203	Liam Turner	2003	20-SEP-26	307	2	950	1800
204	Ava Martinez	2005	18-SEP-26	309	1	420	420
205	Noah Bennett	2006	17-SEP-26	302	2	490	980
206	Mia Johnson	2007	16-SEP-26	308	1	1350	1350
207	Ethan Brooks	2009	11-SEP-26	310	1	2600	2600
209	Oliver Reed	2010	21-SEP-26	306	3	1600	3200

11 rows selected.

8. Customer-wise Order Summary

The output shows each customer along with the number of orders placed and the total amount spent. It summarises each buyer's ordering activity and total purchase value on the marketplace.

```

SELECT
c.Customer_ID,
c.Customer_Name,
COUNT(o.Order_ID) AS Total_Orders,
SUM(o.Total_Amount) AS Total_Amount
FROM Customer c
JOIN Orders o
ON c.Customer_ID = o.Customer_ID
GROUP BY
c.Customer_ID,
c.Customer_Name
ORDER BY
c.Customer_ID;

```

CUST_ID	CUSTOMER_NAME	TOTAL_ORDERS	TOTAL_AMOUNT
201	Emma Wilson	3	3850
202	Sophia Clark	1	890
203	Liam Turner	1	1800
204	Ava Martinez	1	420
205	Noah Bennett	1	980
206	Mia Johnson	1	1350
207	Ethan Brooks	1	2600

CUST_ID	CUSTOMER_NAME	TOTAL_ORDERS	TOTAL_AMOUNT
209	Oliver Reed	1	3200

8 rows selected.

commit;

Commit Completed.